Introduction

Every so often LeetCode hands you a problem that sounds harder than it is, and 3069, Distribute Elements Into Two Arrays I, is a textbook example. The name alone — "distribute" — nudges a lot of people toward heaps, priority queues, or a sort step before they've even read the constraints. That instinct is worth examining, because it's the same instinct that costs candidates time in real interviews: reaching for a heavier tool before checking whether the problem already handed you a shortcut.

This one matters less for its difficulty and more for what it tests. Interviewers use problems like this to see whether you actually read the rule before writing code, or whether you pattern-match to "distribute numbers" → "sort and split" → wrong answer. Once you see the trick, the implementation is almost trivial. Getting there is the real skill, and it's a skill that transfers directly to harder greedy and simulation problems later in your prep.

Problem Overview

You're given an array of `n` distinct integers, 1-indexed conceptually but processed in order. You need to distribute every number into one of two arrays, `arr1` and `arr2`, following one rule:

- The first number always goes into `arr1`.
- The second number always goes into `arr2`.
- For every number after that, look at the **last element currently in `arr1`** and the **last element currently in `arr2`**. Whichever one is bigger, the new number joins that array.

Once every number has been placed, the answer is `arr1` concatenated with `arr2`, in that order.

The constraint that trips people up is subtle: this is not about the size of the piles, and it's not about the numbers overall — it's only ever about the two most recent "tail" values. Nothing else in either array matters for the decision.

Example

Take `nums = [2, 1, 3]`.

- `2` is the first number, so it seeds `arr1`. `arr1 = [2]`.
- `1` is the second number, so it seeds `arr2`. `arr2 = [1]`.

- `3` is next. Compare tails: `arr1`'s last value is `2`, `arr2`'s last value is `1`. `2 > 1`, so `arr1` wins and `3` joins it. `arr1 = [2, 3]`.

Final answer: `arr1 = [2, 3]`, `arr2 = [1]`, concatenated as `[2, 3, 1]`.

Now compare that to what happens if you sort first. Sorted, `nums` becomes `[1, 2, 3]`, and splitting evenly gives `arr1 = [1, 2]`, `arr2 = [3]` — a completely different, and wrong, result. Sorting throws away the one thing this problem actually cares about: the order the numbers arrive in. The rule is defined entirely in terms of arrival order and running tail comparisons, so any transformation that disturbs order breaks it.

A second example makes the mechanics clearer: `nums = [5, 4, 3, 8]`.

- `5` seeds `arr1`. `arr1 = [5]`.
- `4` seeds `arr2`. `arr2 = [4]`.
- `3` compares: `arr1`'s last is `5`, `arr2`'s last is `4`. `5` wins, so `3` joins `arr1`. `arr1 = [5, 3]`.
- `8` compares: `arr1`'s last is now `3`, `arr2`'s last is `4`. `4` wins, so `8` joins `arr2`. `arr2 = [4, 8]`.

Final: `arr1 = [5, 3]`, `arr2 = [4, 8]`, concatenated as `[5, 3, 4, 8]`.

Intuition

The mental trap here is that "distribute into two groups" sounds like a partitioning problem, and partitioning problems often want you to sort first so you can split evenly or by rank. But read the rule again: it never says anything about the final size of each pile, the sum, or the overall value distribution. It only ever asks "which array's most recent number is bigger, right now, at this exact moment in the sequence?"

That's a strong hint. A decision that depends only on the two most recent values, and never on anything further back, is a signal that you don't need a data structure that gives you sorted order, minimum/maximum across the whole set, or random access into the middle. You need exactly one thing, fast: the last element of two growing collections.

Once you frame it that way, the problem stops being about "distribution strategy" and becomes a straightforward simulation: walk the array once, in order, and make one comparison per element. Whatever list is winning at each step gets the append.

This is a useful pattern to internalize for interviews generally — before reaching for a heap, a sorted structure, or a two-pass approach, check whether the rule you're implementing only ever needs the *most recent* state. If it does, a plain pass with simple containers is usually enough, and it's almost always faster to write and easier to reason about than the "advanced" version.

Brute Force Solution

There isn't really a meaningfully different "brute force" here in the traditional sense — this problem's structure basically forces you toward the simulation from the start unless you try something misguided like sorting first.

Idea: Simulate the exact rule as stated: seed the two arrays with the first two elements, then for each subsequent element check both tails and append accordingly.

Algorithm: 1. Initialize `arr1` with `nums[0]`, `arr2` with `nums[1]`. 2. For each remaining number, compare `arr1[-1]` and `arr2[-1]`. 3. Append to whichever array has the bigger last value. 4. Return `arr1 + arr2`.

Advantages: Simple, direct translation of the problem statement, no wasted work.

Disadvantages: None really — this "brute force" already is the optimal approach because the problem doesn't leave room for a smarter algorithm underneath it. There's no redundant computation to eliminate.

Time complexity: O(n) — one pass, one comparison per element. **Space complexity:** O(n) — for the two output arrays.

Optimal Solution

Because the naive simulation is already optimal, the "optimal solution" and the brute force are the same algorithm. The only real design decision is the data structure choice.

You need constant-time access to the last element of two growing collections, and constant-time append. A plain Python list gives you both: `list[-1]` for the tail, `list.append(x)` for insertion, both O(1) amortized. There's no reason to reach for a deque, heap, or any specialized structure — none of them offer anything a list doesn't already give you here, and they'd add overhead for no benefit.

Step	arr1	arr2	New number	Comparison	Goes to
seed	[5]	—	—	—	arr1
seed	[5]	[4]	—	—	arr2

Step	arr1	arr2	New number	Comparison	Goes to
3	[5]	[4]	3	5 > 4	arr1
4	[5, 3]	[4]	8	3 < 4	arr2

Python Code

```
from typing import List


class Solution:
    def resultArray(self, nums: List[int]) -> List[int]:
        arr1 = [nums[0]]
        arr2 = [nums[1]]

        for num in nums[2:]:
            if arr1[-1] > arr2[-1]:
                arr1.append(num)
            else:
                arr2.append(num)

        return arr1 + arr2


def demo() -> None:
    assert Solution().resultArray([2, 1, 3]) == [2, 3, 1]
    assert Solution().resultArray([5, 4, 3, 8]) == [5, 3, 4, 8]
    print("all assertions passed")


if __name__ == "__main__":
    demo()
```

Code Walkthrough

- `arr1 = [nums[0]]` and `arr2 = [nums[1]]` handle the seeding rule directly — the first number always starts `arr1`, the second always starts `arr2`. This also sidesteps a subtle edge case: if you tried to loop from index 0 with empty lists, your very first comparison would hit an empty list and crash. Seeding first means the loop body never has to special-case "what if a pile is empty."
- `for num in nums[2:]` iterates over everything after the two seed values, in original order — order is the entire point of this problem, so nothing here reorders or re-groups the input.
- `arr1[-1] > arr2[-1]` is the whole decision rule: negative indexing gives O(1) access to the last element without searching.

- `arr1.append(num)` / `arr2.append(num)` grows whichever pile is currently ahead.
- `return arr1 + arr2` concatenates the two finished piles into the single output array the problem expects.

Dry Run

Using `nums = [5, 4, 3, 8]`:

1. `arr1 = [5]`, `arr2 = [4]` after seeding.
2. `num = 3`: `arr1[-1] = 5`, `arr2[-1] = 4`. `5 > 4` → append to `arr1`. `arr1 = [5, 3]`.
3. `num = 8`: `arr1[-1] = 3`, `arr2[-1] = 4`. `3 > 4` is false → append to `arr2`. `arr2 = [4, 8]`.
4. Return `arr1 + arr2 = [5, 3, 4, 8]`.

Matches the expected result.

Complexity Analysis

Time: O(n). Every element in `nums` is visited exactly once, and each visit does a fixed amount of work — one comparison, one append. There's no nested loop, no re-scanning, and no hidden cost in `arr[-1]` (Python lists give O(1) access to any index, including negative ones).

Space: O(n). The two output arrays together hold every element from the input exactly once. There's no auxiliary structure beyond that.

Both bounds are tight — you can't do better than O(n) time because every element must be read and placed at least once, and you can't do better than O(n) space because the problem requires you to return all n elements.

Alternative Solutions

The only "alternative" worth mentioning is the wrong one: sorting `nums` first and splitting it some way. As shown in the example above, this produces a different result because the rule is order-dependent, not value-dependent. It's included here specifically because it's the most common mistake, not because it's a valid approach — there is no legitimate alternative algorithm that beats or matches the direct simulation, since the simulation is already both correct and asymptotically optimal.

Edge Cases

- **Minimum size input:** The problem guarantees at least 2 elements (since both arrays need a seed), so you don't need to guard against `nums` having fewer than 2 elements — but if you were to receive one, indexing `nums[1]` would throw, which is the correct behavior for an invalid input under the problem's constraints.
- **Two-element input:** With exactly `[a, b]`, the loop body never executes; the result is just `[a] + [b]`.
- **Strictly increasing sequence:** Since values are distinct and each new max will always exceed the current tail of one array, watch how ties never occur here — no special tie-break logic is exercised.
- **Strictly decreasing sequence:** Every new number is smaller than both tails, so the comparison still resolves cleanly since it only checks relative size, not whether the new number itself is larger than anything.
- **All numbers distinct is guaranteed by constraints** — but if you ever reused this pattern where equal tail values were possible, you'd need to decide a tie-break rule; this problem doesn't require one.

Common Mistakes

1. **Sorting `nums` before processing.** This destroys the arrival order the rule depends on and produces a different, wrong result.
2. **Comparing the full arrays instead of just the last elements** — e.g., comparing sums or lengths of `arr1` and `arr2`. The rule only cares about the two most recent tail values.
3. **Forgetting to seed both arrays before the loop**, then hitting an index error or wrong comparison on an empty list.
4. **Off-by-one in the loop start**, iterating from `nums[1:]` or `nums[0:]` instead of `nums[2:]`, which reprocesses an already-seeded number.
5. **Returning `arr2 + arr1` instead of `arr1 + arr2`** — the order of concatenation matters for the expected output.
6. **Using a data structure that doesn't give O(1) last-element access**, like a set, which would drop ordering information entirely.

Interview Questions

- What would change if the rule allowed ties (equal tail values)? How would you decide where the number goes?
- Could you solve this without seeding — i.e., handle it with a single uniform loop from index 0? What extra condition would you need?

- How would the approach change if you needed to support three arrays instead of two?
- Can you do this in O(1) extra space beyond the output arrays?
- What's the difference between this problem and a "greedy load balancing" problem where you assign to the currently smaller pile instead?

Similar Problems

- **LeetCode 2894, Divisible and Non-divisible Sums Difference** — another single-pass simulation problem where reading the rule carefully beats reaching for a heavier structure.
- **LeetCode 1046, Last Stone Weight** — also compares "tail" or "top" values repeatedly, though it needs a max-heap since it's not order-preserving.
- **LeetCode 921, Minimum Add to Make Parentheses Valid** — another greedy single-pass problem where only the most recent state matters at each step.
- **LeetCode 3070, Distribute Elements Into Two Arrays II** — the harder follow-up version of this exact problem, where the comparison rule gets more complex.

Key Takeaways

The lesson here isn't really about this specific problem — it's about recognizing when a rule only depends on recent state. Once you notice that a decision only ever looks at the last value in two growing collections, you know you need O(1) tail access and nothing fancier. Seed first to avoid special-casing empty containers, then loop once, compare, and append. That's the entire solution, and it's a pattern worth carrying into other greedy and simulation problems.

Watch the Video

For the full walkthrough with the trace through both examples, watch the video here: {LINK}

About the Series

This breakdown is part of Fun with Learning Technology, a series where Kai works through a LeetCode problem a day in Python, focused on building the intuition behind each solution rather than just memorizing the code.

Call To Action

If this cleared things up, like the video on YouTube and subscribe so tomorrow's problem lands in your feed. For the full written breakdown and future deep dives, subscribe to the Substack. Drop a comment with your own solution or a problem you'd like covered next, and share this with anyone prepping for interviews.

Quick Review

Two-pile distribution problem: what's the tell for this pattern?

Compare-last-element-of-each-pile pattern: when placement depends on comparing new item to the tail of two growing groups.

Time complexity of distributing n elements one-by-one by comparing pile tails?

O(n) — one constant-time comparison and append per element after the fixed 2-element seed.

Space complexity for arr1/arr2 in this distribution approach?

O(n) — both output arrays together hold all n elements; no extra structures needed.

Common bug when seeding arr1 and arr2 from nums?

Forgetting arr1=[nums[0]], arr2=[nums[1]] before the loop, then comparing against an empty array on the first real element.

How does this differ from a sorting problem despite comparing values?

You never reorder elements within a pile — you only decide which pile an element joins, based on its last-element comparison.

Interview follow-up: what if arr1 and arr2 tie on last element?

Tie goes to the pile with fewer elements (or per problem spec — clarify the tie-break rule before coding).
